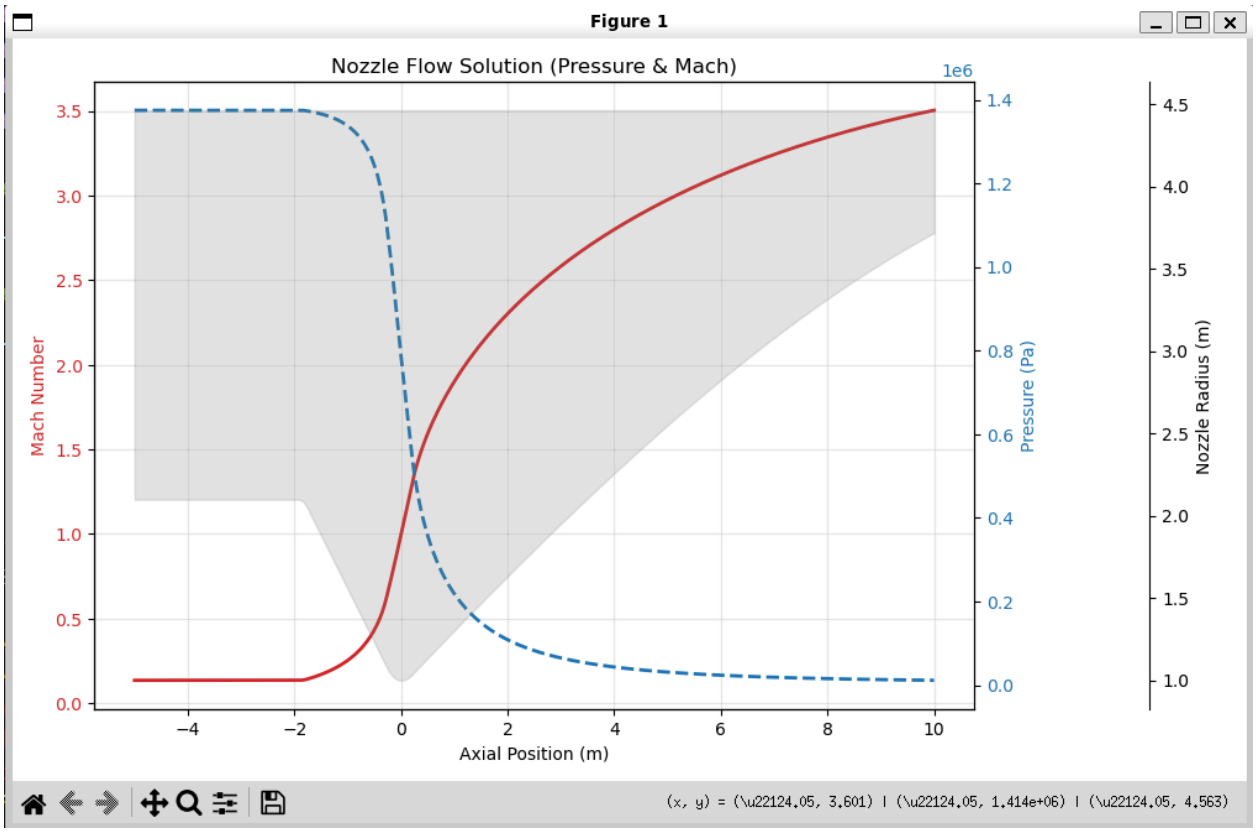
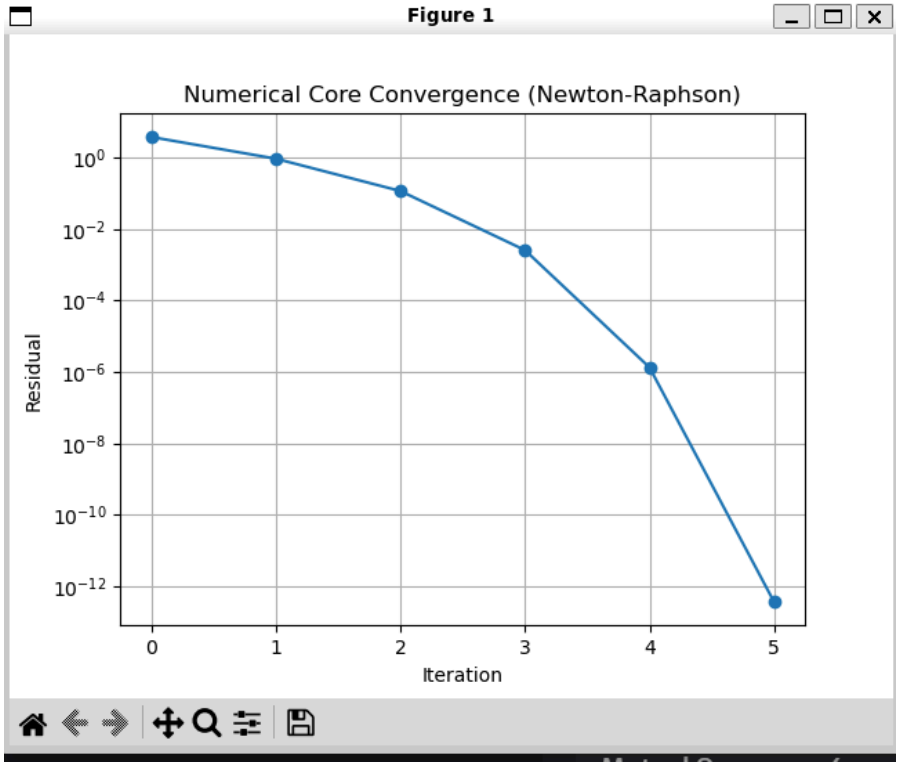
Overall Project Goal: Improve upon ASTE475 final project and convert to python, make it into a downloadable python library

I had a rough idea to translate and expand upon 475 final project. Initial plan was get something that worked and then make it good.

These were the first plots that answered core requirements of mini project

First plot from vibe code only -





I did not like/understand how gemini was organizing the code so decided to better define my requirements and then start fresh

PROJECT REQUIREMENTS

Project Objective: Extension of ASTE475 Final Project and conversion from matlab into python library that can be used to assist in designing ideal nozzles / sizing the required amount of propellant to go to a certain target altitude with given engine parameters

Step 1 - minimum product: Give the user to all 475 equations including normal/oblique shocks, expansion fans, and all normal ideal isentropic flow equations

Step 2 - Reasonable Target: Recreate all logic to allow the user to give same input files as 475 project and make all the same graphs to allow user to analyze specific nozzle geometry

Step 3 - Core project goal: Animated plot of nozzle pressure as rocket increases in altitude and pressure decreases

Step 4 - Push goal: Tool to help with design of ideal nozzle values

Numerical core: Baseline of the same bisection solver written for the ASTE475 project with attempt to improve by creating a hybrid newton-raphson solver

- Verification of the method through a residual history plot to demonstrate decrease in error magnitude over time, faster than bisection solver
- Hybrid due to singularity risk at throat

Pathway to verification

Version control with Github

PDF report

1. **+Project overview:** What you built, why it matters, what question you're answering
2. **+Numerical method(s):** Equations, discretization/algorithm (pseudocode is fine), stability/accuracy considerations/limitations
3. **+Implementation notes:** Key design choices, file/directory layout, how to run (similar to README)
4. **~Progress log:** Dated entries or milestones (what you tried, what broke, what you changed, what you learned)
5. **~Verification/validation evidence:** Even just partial is okay here. Show what you checked, what you were thinking about, and what remains
6. **Results:** Plots/tables/screenshots—interpret what's shown (and feel free to ask me for help if necessary)
7. **Reflection/next steps:** What would you do if you had more time? What did you learn?
8. **LLM transcripts:** Include it all at the end, each in

Numerical Methods

475 final project used a very simple bisection solver that was slow and could not converge to as quickly as more advanced solvers. For this 1D solver significantly more complicated numerical method were not necessary, so I decided on an incremental improvement with a Newton solver that had a backup bisection in case of divergence.

Primary goal to solve this equation:

$$f(M) = \frac{A}{A^*} - \frac{1}{M}\left[\left(\frac{2}{\gamma+1}\right)\left(1+\frac{\gamma-1}{2}M^2\right)\right]^{\frac{\gamma+1}{2(\gamma-1)}} = 0$$

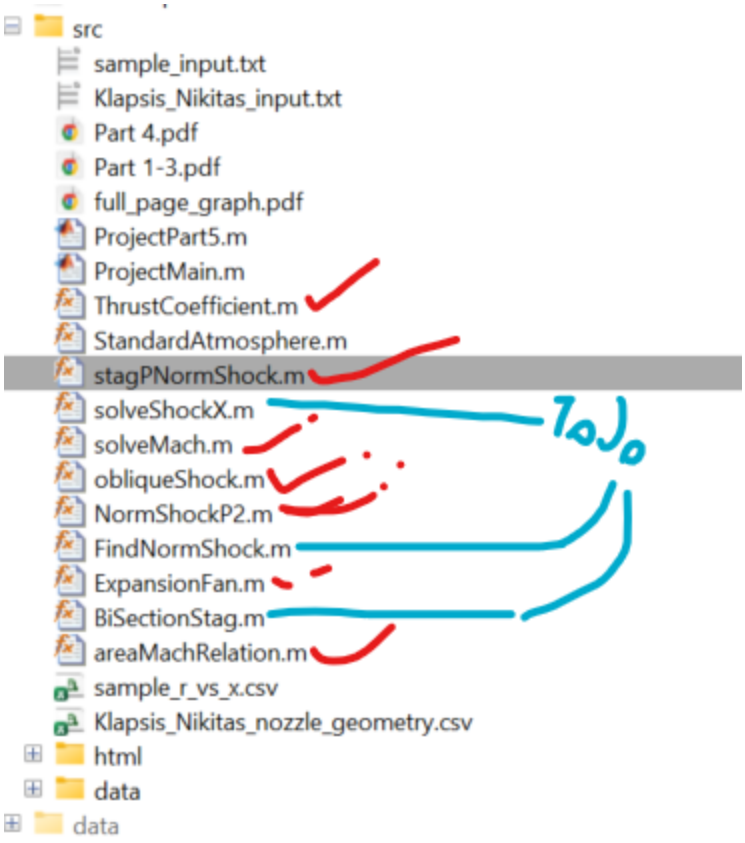
Utilizing its derivative to increase the efficiency of solving it

$$f'(M) = \frac{A}{A^*} \cdot \frac{M^2-1}{M\left(1+\frac{\gamma-1}{2}M^2\right)}$$

Psuuedo code:

```
FUNCTION Solve(target_func, derivative_func, guess, min_bound, max_bound):
    x_current = guess
    Initialize error_history

    FOR iteration = 1 TO max_iterations:
        f_val = target_func(x_current)
        f_prime = derivative_func(x_current)

        # 1. Check Convergence
        IF abs(f_val) < tolerance:
            RETURN x_current

        # 2. Attempt Newton Step
        # x_new = x - f(x) / f'(x)
        step = f_val / f_prime
        x_new = x_current - step

        # 3. Stability Check (The Hybrid Logic)
        # If Newton step shoots outside physical bounds [min, max],
        # reject it and use Bisection (midpoint) instead.
        IF (x_new <= min_bound) OR (x_new >= max_bound):
            x_new = (min_bound + max_bound) / 2.0

        # 4. Bracket Update (Standard Bisection Logic)
        # Tighten the bounds to ensure global convergence
        IF target_func(min_bound) * target_func(x_new) < 0:
            max_bound = x_new
        ELSE:
            min_bound = x_new

        x_current = x_new

    RETURN x_current (or raise ConvergenceError)
```

STABILITY: At the nozzle the derivative approaches 0 and therefore may cause a crash if f / f’ goes to infinity, therefore a backup bisection solver was added



Completed all of the following function in one main gas dynamics module, will now focus on combining everything into analyzing shocks in a nozzle

Manual review/rewrite of a lot of nozzle code

```
11  class NozzleAnalyzer:
12      def __init__(self, inputs: dict, geometry_df):···
35
36      def solve_isentropic(self):···
77
78      def detect_shock_type(self):···
119
120     def _find_normal_shock_location(self, tolerance=0.05):···
178
179     def _calculate_normal_shock_flow(self, shock_idx):···
246
247     def plot_results(self):···
307
308     def _generate_shock_summary(self):···
338
339     def _plot_nozzle_contour(self, ax):···
```

After manually reviewing all of gas dynamics, had to tweak nozzle file a lot to get desired output. Mix of gemini, plus copilot claude, plus manual code to get it into working state. After research it seemed my original matlab method of doing normal shocks by pretending the shock was at the exit and working backwards from there was flawed, so utilized a new method that iteratively checked every point for a normal shock that would result in the correct ambient pressure. Not sure on the correctness of this, will need to review.

Animation was done hastily at the end and therefore may have issues.

Look at readme / index for more indepth information about the format of the code.

VALIDATION

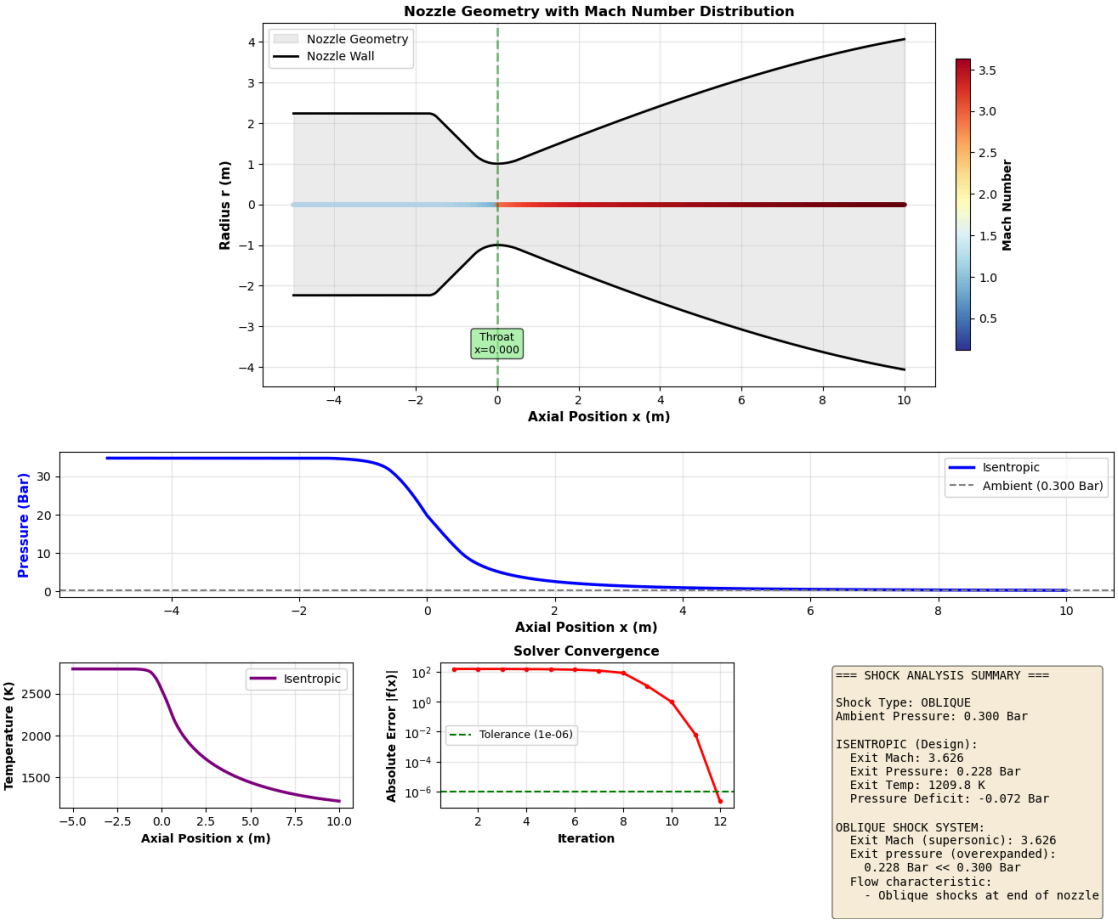
Verified mathematical results against matlab code, did not have time to implement formal pytest methods

Results

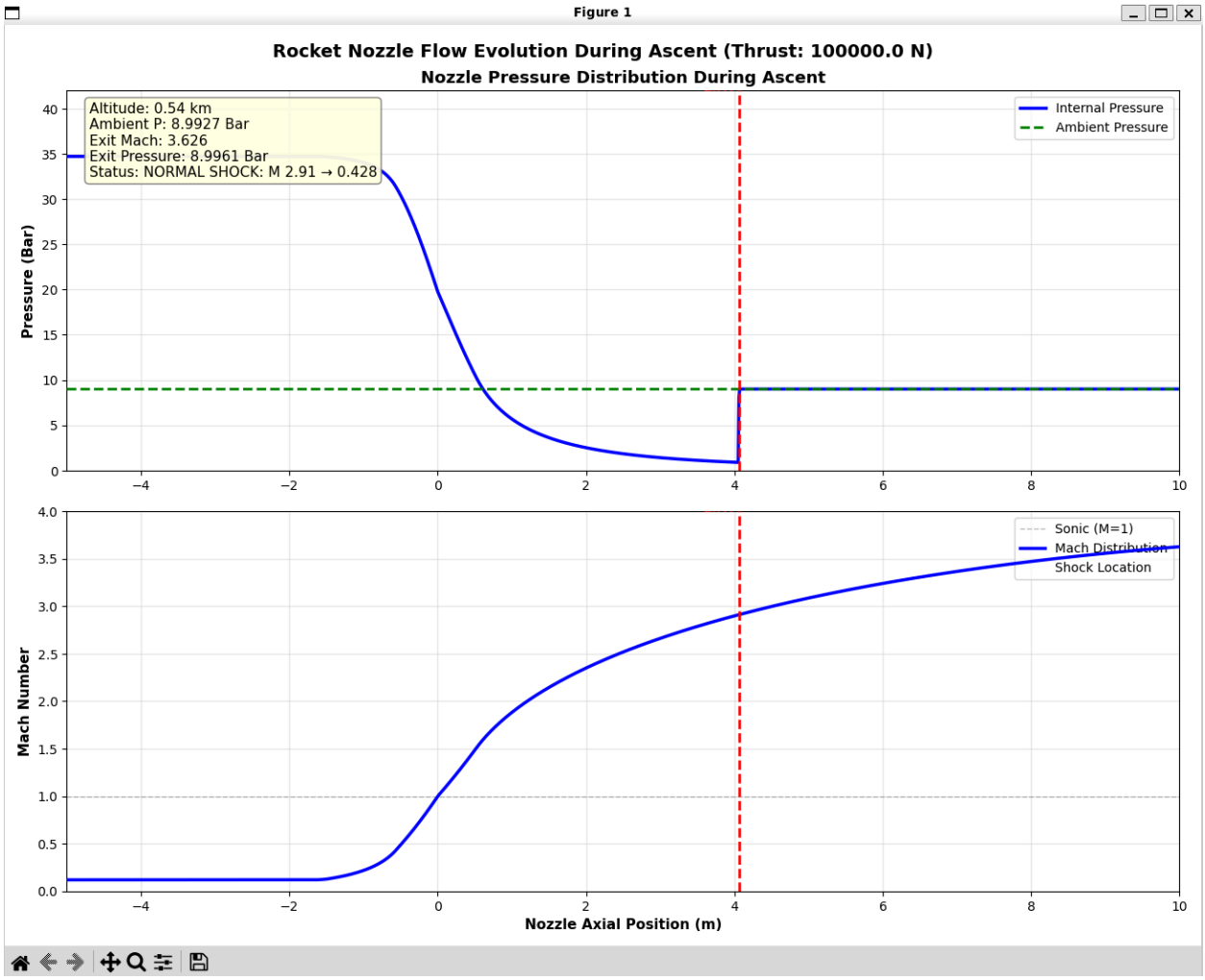
Mathematical functions:

```
(aste404) nklapsis@Nikitas-Laptop:/mnt/c/Users/ndkla/OneDrive/Desktop/2025 Fall Classes/ASTE404/ASTE404FinalProject/tests$ pyt
 test_step1.py
=== STEP 1 VERIFICATION ===

[1] Testing Solver on Isentropic Relation...
   Target A/A*=2.0. Calculated Mach: 2.1972

[2] Testing Normal Shock...
   M1=2.0 -> M2=0.5774

[3] Testing Expansion Fan...
   M1=2.0, Theta=10 -> M2=2.3849

[4] Testing Oblique Shock (Theta-Beta-M)...
   M1=2.0, Theta=10 -> Beta=39.3139

All Step 1 tests passed successfully.
(aste404) nklapsis@Nikitas-Laptop:/mnt/c/Users/ndkla/OneDrive/Desktop/2025 Fall Classes/ASTE404/ASTE404FinalProject/tests$
```

Nozzle Simulation:



Animated plot (still frame shown)



9. **Reflection/next steps:**

Implement part 4 for actual design with propellant properties, flush out mathematical toolbox

I want you to help me write a python library for my aste404 final mini project that follows these instructions. I will be adapting another project I wrote in matlab and expanding upon it which I will provide you. I've provided most of the files in the old matlab project and in the next prompt I will give you the remaining files as well as some other files that will help

These are the remaining matlab files and csv file that defines the nozzle geometry and here is hw5 instructions of how to create the python library

I've given you the lectures to the class, see if you can improve the code. Maybe substituting bisection solver with something else based on these lectures

I want this to be a tool where the user gives engine characteristics like thrust, isp, chamber pressure, chamber temperature, etc and inputs a desired altitude and the program determines the ideal constract ratio and expansion ratio as well as necessary amount of propellants for a given mix like methalox

For full clarity I want to keep the utility of the original project where a user can enter a geometry in the format provided with info in the txt provided and get a plot of pressure, shocks, etc. I just want additional utility that a user can either select a propellant mix like the code you provided or provide their own values with engine design values and desired altitude to get a ideal nozzle geometry and propellant sizing

show me what my file tree should look like with everything and confirm that with these inputs I can do the full analysis section

give me contents for the toml file and the __ideal__ file

I'm following this homework template and this gives me an error:

nklapsis@Nikitas-Laptop:/mnt/c/Users/ndkla/OneDrive/Desktop/2025 Fall Classes/ASTE404/ASTE404FinalProject$ mamba install hatchling build

conda-forge/noarch 23.8MB @ 11.0MB/s 2.1s

```
conda-forge/linux-64                                49.5MB @   8.0MB/s  6.2s


Pinned packages:
  - python=3.12
error    libmamba Could not solve for environment specs
    The following package could not be installed
    └─ build =* * does not exist (perhaps a typo or a missing channel).
critical libmamba Could not solve for environment specs
```

now how do I verify that the code works, I also want a input subfolder either in src or in the main file

```
/home/nklapsis/miniforge3/envs/aste404/bin/python: Error while finding module specification for 'aste404_miniproject.main' (ModuleNotFoundError: No module named 'aste404_miniproject')
(aste404) nklapsis@Nikitas-Laptop:/mnt/c/Users/ndkla/OneDrive/Desktop/2025 Fall Classes/ASTE404/ASTE404FinalProject$ pip list | grep aste404
aste404_miniproject          0.1.0         /mnt/c/Users/ndkla/OneDrive/Desktop/2025 Fall Classes/ASTE404/ASTE404FinalProject
(aste404) nklapsis@Nikitas-Laptop:/mnt/c/Users/ndkla/OneDrive/Desktop/2025 Fall Classes/ASTE404/ASTE404FinalProject$ python -m aste404_miniproject.main analyze \
    --input inputs/Klapsis_Nikitas_input.txt \
    --geometry inputs/Klapsis_Nikitas_nozzle_geometry.csv \
    --diagnose
```

```
/home/nklapsis/miniforge3/envs/aste404/bin/python: Error while finding module specification for 'aste404_miniproject.main' (ModuleNotFoundError: No module named 'aste404_miniproject')

(aste404) nklapsis@Nikitas-Laptop:/mnt/c/Users/ndkla/OneDrive/Desktop/2025 Fall Classes/ASTE404/ASTE404FinalProject$ python -m aste404_miniproject.main analyze --input inputs/Klapsis_Nikitas_input.txt     --geometry inputs/Klapsis_Nikitas_nozzle_geometry.csv     --diagnose

/home/nklapsis/miniforge3/envs/aste404/bin/python: Error while finding module specification for 'aste404_miniproject.main' (ModuleNotFoundError: No module named 'aste404_miniproject')

(aste404) nklapsis@Nikitas-Laptop:/mnt/c/Users/ndkla/OneDrive/Desktop/2025 Fall Classes/ASTE404/ASTE404FinalProject$ ls -R

.:

README.md  dist  inputs  pyproject.toml  src

./dist:

aste404_miniproject-0.1.0-py3-none-any.whl  aste404_miniproject-0.1.0.tar.gz

./inputs:

Klapsis_Nikitas_input.txt  Klapsis_Nikitas_nozzle_geometry.csv

./src:

aste404_miniproject.egg-info  aste404finalproject

./src/aste404_miniproject.egg-info:

PKG-INFO  SOURCES.txt  dependency_links.txt  entry_points.txt  requires.txt  top_level.txt
```

./src/aste404finalproject:

__init__.py design.py gas_dynamics.py main.py nozzle.py propellants.py solver.py utils.py

(aste404) nklapsis@Nikitas-Laptop:/mnt/c/Users/ndkla/OneDrive/Desktop/2025 Fall Classes/ASTE404/ASTE404FinalProject$

type shit gemini

type shit means good job

can you now help me test out the design part of the package

I want the analysis code to be able to detect and plot shocks given different ambient pressures to make a plot like this

it did not add the shocks

lets take a step back and go line by line through the code so I fully understand what it does

explain what parsers and arguements here do: # MODE 1: ANALYZE

    p1 = sub.add_parser('analyze', help='Analyze existing nozzle')

    p1.add_argument('--input', required=True)

    p1.add_argument('--geometry', required=True)

    p1.add_argument('--diagnose', action='store_true')

    # NEW FLAG HERE

    p1.add_argument('--sweep', action='store_true', help='Plot pressure distributions for various shock locations')

    # MODE 2: DESIGN

p2 = sub.add_parser('design', help='Design new engine')

p2.add_argument('--thrust', type=float, required=True)

p2.add_argument('--pc', type=float, default=50.0, help='Bar')

p2.add_argument('--alt', type=float, default=0.0, help='km')

p2.add_argument('--time', type=float, default=10.0)

p2.add_argument('--prop', type=str, default='methalox')

Describe the numerical core of this project ensuring you answer the project requirements: Numerical core

• At least one "real" numerical method (e.g. finite differences, iterative linear solvers, time integration, root finding, interpolation, Monte Carlo estimation, optimization, etc.)

– The method need not be implemented from scratch (i.e. you can use external libraries)

• You must explain why the chosen method is appropriate and what its limitations/stability constraints are

I want to take a step back and build the project up bit by bit, ensuring I understand and agree with you for the full structure of it as we build up. Here is my plan to do that and I want you to first help me just with step 1 for now: Project Objective: Extension of ASTE475 Final Project and conversion from matlab into python library that can be used to assist in designing ideal nozzles / sizing the required amount of propellant to go to a certain target altitude with given engine parameters

Step 1 - minimum product: Give the user to all 475 equations including normal/oblique shocks, expansion fans, and all normal ideal isentropic flow equations

Step 2 - Reasonable Target: Recreate all logic to allow the user to give same input files as 475 project and make all the same graphs to allow user to analyze specific nozzle geometry

Step 3 - Core project goal: Animated plot of nozzle pressure as rocket increases in altitude and pressure decreases

Step 4 - Push goal: Tool to help with design of ideal nozzle values

Numerical core: Baseline of the same bisection solver written for the ASTE475 project with attempt to improve by creating a hybrid newton-raphson solver

Verification of the method through a residual history plot to demonstrate decrease in error magnitude over time, faster than bisection solver

Hybrid due to singularity risk at throat

(aste404) nklapsis@Nikitas-Laptop:/mnt/c/Users/ndkla/OneDrive/Desktop/2025 Fall Classes/ASTE404/ASTE404FinalProject/tests$ python test_step1.py

Traceback (most recent call last):

  File "/mnt/c/Users/ndkla/OneDrive/Desktop/2025 Fall Classes/ASTE404/ASTE404FinalProject/tests/test_step1.py", line 1, in <module>

    from src.aste404_miniproject.gas_dynamics import (

ModuleNotFoundError: No module named 'src'

python test_step1.py

Traceback (most recent call last):

  File "/mnt/c/Users/ndkla/OneDrive/Desktop/2025 Fall Classes/ASTE404/ASTE404FinalProject/tests/test_step1.py", line 1, in <module>

    from aste404_miniproject.gas_dynamics import (

File "/mnt/c/Users/ndkla/OneDrive/Desktop/2025 Fall Classes/ASTE404/ASTE404FinalProject/src/aste404_miniproject/__init__.py", line 10, in <module>

from .nozzle import NozzleAnalyzer

File "/mnt/c/Users/ndkla/OneDrive/Desktop/2025 Fall Classes/ASTE404/ASTE404FinalProject/src/aste404_miniproject/nozzle.py", line 3, in <module>

from .solver import NewtonSolver

ImportError: cannot import name 'NewtonSolver' from 'aste404_miniproject.solver' (/mnt/c/Users/ndkla/OneDrive/Desktop/2025 Fall Classes/ASTE404/ASTE404FinalProject/src/aste404_miniproject/solver.py)

what is the arguement for having all the ideal rocket equations in a single file vs having a seperate file for each function like I did for my matlab project

For step two ensure everything is as readable as possible and walk me through the process step by step

why is nozzle a class

I want to be able to plot the drop in pressure after a shock if there is a normal shock inside the nozzle

if there is a normal shock in the nozzle does the flow have to exit at ambient pressure or is that no longer the case

please provide a source that shows that after a normal shock the pressure must return to ambient

see if this file is correct at finding normal shocks

The logic in your `nozzle.py` file is fundamentally sound and correctly implements the "Shooting Method" for finding normal shocks.

Here is a breakdown of why it works, along with a few minor "watch-outs" to ensure it's robust.

1. The Logic is Correct

You are strictly following the physics we discussed:

- **The Scanner (`_find_normal_shock_location`):**
  - It iterates through the divergent section (`throat_idx + 1` to end).
  - It places a "virtual shock" at each point.
  - It calculates the pressure loss (P
  - 02
  -
  -) and area change (A
  - 2
  - ∗
  -
  -).
  - It solves the **subsonic** flow from that shock to the exit.
  - It checks if the calculated P
  - *exit*
  -
  - matches the input P
  - *ambient*
  -
  - .
  - This is the standard textbook approach for 1D nozzle flows with internal shocks.
- **The Calculator (`_calculate_normal_shock_flow`):**
  - Once the best index is found, it effectively "freezes" that solution into the `self.M_post_shock` arrays for plotting.
  - It correctly updates the stagnation pressure P
  - 0
  -
  - for the downstream section, which is the most common mistake students make. You handled it correctly with `P0_new = Pc * P02_P01`

2. Visualization of What Your Code Does

Your code is trying to replicate the behavior seen in standard gas dynamics plots.

- **Isentropic Expansion:** The flow follows the design curve (blue line in typical plots).
- **The Jump:** At the `shock_idx`, pressure jumps up instantaneously.
- **Subsonic Diffusion:** After the shock, the pressure rises gradually to meet the ambient pressure at the exit.

Your code produces exactly this behavior:

Python

```
# From your code:
# Pressure jumps up
P2 = P1 * P2_P1
# Then flow effectively compresses/diffuses to exit
func = lambda m: area_mach_relation(m, g) - ratio
```

3. Two Minor "Watch-outs"

While the logic is correct, here are two edge cases to keep in mind:

A. The "Weak Shock" Instability near M=1 In `_find_normal_shock_location`, you have this check:

Python

```
if M_before <= 1.0:
    continue
```

Recommendation: Change this to `if M_before <= 1.05:`.

- **Why?** Normal shock relations become numerically unstable very close to Mach 1.0. A shock at M=1.001 is basically just a sound wave. The solver might struggle or produce noise there. A small buffer prevents this.

B. The "Second Throat" Problem If your nozzle has a geometry that contracts *after* the shock location (e.g., if the shock happens early in the bell and the bell has a weird inflection), the flow could theoretically re-choke (reach M=1 again).

- **Your Code:** Currently has a `try/except` block that falls back to `0.01` or `continue`
- **Verdict:** This is acceptable for this project because standard rocket nozzles (conical or bell) are monotonic—they only expand. The area A always gets bigger, so the flow will never re-choke. You are safe.

Summary

Yes, this file is correct. It effectively scans for the unique location where the shock strength is exactly right to compress the flow back to ambient pressure by the exit plane.

Next Step: You can run this file using your `main.py` or the test script. If you provide an input file with a high ambient pressure (like 1 atm) and a high expansion ratio nozzle, you should see the shock appear in the plots.

which lecture talks about the newton-raphson solver

I do not have access to your specific course syllabus or lecture list to tell you which exact lecture number covers the Newton-Raphson method.

However, based on the `aste404_miniproject` structure you have built, the **Newton-Raphson solver** is the numerical core used to solve the nonlinear area-Mach number relation.

In the context of the ASTE 404 course material, this topic is typically found in:

- **Compressible Flow / Gas Dynamics Modules:** Specifically the section on **Isentropic Flow** or **Area-Mach Relation**. This is because solving for Mach number (M) given an Area Ratio (A/A
- $*$
-) is the classic engineering problem that requires a numerical root-finding method like Newton-Raphson or Bisection.
- **Numerical Methods Introduction:** If your course has a specific module on computational tools, it would be introduced there as a standard algorithm for finding roots of equations of the form $f(x)=0$.

If you are looking for where *your code* implements this, it is located in: `src/aste404_miniproject/solver.py`.

Now complete step 3 that iteratively shows the nozzle conditions as the rocket ascends in the atmosphere

This is the "Showstopper" of your project. We will create an animation that simulates the rocket launch from **Sea Level (0 km)** to **Space (30 km)**.

As the rocket rises, the ambient pressure (P

amb

) drops. You will see the normal shock start deep inside the nozzle and slowly "push" its way out until it pops out of the exit, leaving you with perfect supersonic flow.

1. Create `src/aste404_miniproject/animation.py`

This file handles the simulation loop. It runs the solver ~100 times (once for each altitude step) and saves the frames for smooth playback.

Python

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from .nozzle import NozzleAnalyzer

class NozzleAnimator:
    def __init__(self, inputs, geometry):
        self.inputs = inputs
        self.geometry = geometry
        # Create a single analyzer instance to reuse
        self.analyzer = NozzleAnalyzer(inputs, geometry)

        # Simulation Settings: 0km to 30km
        self.altitudes = np.linspace(0, 30000, 150)
        self.frames = []

    def _get_ambient_pressure(self, altitude_m):
```

```
        """
        Simple Exponential Atmosphere Model.
        P = P0 * exp(-h / ScaleHeight)
        """
        P0 = 101325.0  # Sea Level Pa
        H = 7200.0     # Scale Height (meters)
        return P0 * np.exp(-altitude_m / H)

    def precalculate(self):
        """
        Runs the numerical solver for every altitude step BEFORE
plotting.
        This ensures the animation runs smoothly without lag.
        """
        print(f"Pre-calculating {len(self.altitudes)} flight
frames...")

        base_inputs = self.inputs.copy()

        for i, alt in enumerate(self.altitudes):
            # 1. Update Ambient Pressure
            current_amb = self._get_ambient_pressure(alt)
            base_inputs['Pb1'] = current_amb

            # 2. Reset and Run Analyzer
            # We create a fresh instance or update the existing
one
            self.analyzer.inputs['Pb1'] = current_amb

            # Solve Base Flow
            self.analyzer.solve_isentropic()

            # Detect and Solve Shocks
            shock_type = self.analyzer.detect_shock_type()

            # 3. Extract Data for Plotting
            frame_data = {
                'alt': alt,
                'P_amb': current_amb,
                'x': self.analyzer.x,
                'shock_x': None
```

```
            }

            if shock_type == 'normal':
                # Use the post-shock pressure array
                frame_data['P'] = self.analyzer.P_post_shock
                if self.analyzer.shock_location is not None:
                    frame_data['shock_x'] =
self.analyzer.x[self.analyzer.shock_location]
            else:
                # Use the standard isentropic pressure array
                frame_data['P'] = self.analyzer.P

            self.frames.append(frame_data)

            # Simple progress bar
            if i % 20 == 0:
                print(f"  > Processed {alt/1000:.1f} km...")

        print("Calculation Complete. Launching Animation...")

    def run(self):
        """Sets up and plays the Matplotlib animation."""
        if not self.frames:
            self.precalculate()

        fig, ax = plt.subplots(figsize=(10, 6))

        # Static Limits (so the axis doesn't jump around)
        ax.set_xlim(self.geometry['x'].min(),
self.geometry['x'].max())
        ax.set_ylim(0, self.inputs['Pc'] / 1e5 * 1.1)

        ax.set_xlabel('Nozzle Position (m)')
        ax.set_ylabel('Pressure (Bar)')
        ax.grid(True, alpha=0.3)

        # Initialize Plot Objects
        line_pressure, = ax.plot([], [], 'b-', lw=2.5,
label='Internal Pressure')
        line_ambient, = ax.plot([], [], 'g--', lw=1.5,
label='Ambient Pressure')
```

```
        shock_line = ax.axvline(0, color='r', ls='-', lw=2,
alpha=0) # Hidden initially

        # Info Text
        stats_text = ax.text(0.02, 0.95, "",
transform=ax.transAxes,
                             bbox=dict(boxstyle="round",
fc="white", ec="gray", alpha=0.9))

        def init():
            return line_pressure, line_ambient, shock_line,
stats_text

        def update(frame):
            # Unpack data
            x = frame['x']
            P = frame['P'] / 1e5  # Convert to Bar
            P_amb = frame['P_amb'] / 1e5
            alt_km = frame['alt'] / 1000.0

            # Update Curves
            line_pressure.set_data(x, P)
            line_ambient.set_data([x[0], x[-1]], [P_amb, P_amb])

            # Update Shock Marker
            if frame['shock_x'] is not None:
                shock_line.set_xdata([frame['shock_x']])
                shock_line.set_alpha(1.0)
                status = "OVEREXPANDED (Normal Shock)"
            else:
                shock_line.set_alpha(0.0)
                if P[-1] > P_amb:
                    status = "UNDEREXPANDED (Clean)"
                else:
                    status = "OVEREXPANDED (Exit Shock)"

            # Update Text
            stats_text.set_text(
                f"Altitude: {alt_km:.1f} km\n"
                f"Ambient P: {P_amb:.2f} bar\n"
                f"Status: {status}"
```

```
            )

            return line_pressure, line_ambient, shock_line,
stats_text

        # Create Animation
        # interval=50 means 50ms per frame (20 fps)
        ani = FuncAnimation(fig, update, frames=self.frames,
                            init_func=init, blit=False,
interval=50)

        plt.title(f"Nozzle Flow Evolution during Ascent (Thrust:
{self.inputs['NominalThrust']} N)")
        plt.legend(loc='upper right')
        plt.show()
```

2. Create `test_step3.py`

This script ties it all together.

Python

```
from aste404_miniproject.utils import load_inputs, load_geometry
from aste404_miniproject.animation import NozzleAnimator

def run_step3():
    print("=== STEP 3: FLIGHT ANIMATION ===")

    # 1. Load your specific homework files
    inputs = load_inputs("inputs/Klapsis_Nikitas_input.txt")
    geo =
load_geometry("inputs/Klapsis_Nikitas_nozzle_geometry.csv")

    # 2. Initialize the Animator
    # We pass the raw inputs and geometry. The animator handles
the loop.
    animator = NozzleAnimator(inputs, geo)

    # 3. Run
```

```
    # This will trigger the pre-calculation and then open the
window.
    animator.run()

if __name__ == "__main__":
    run_step3()
```

3. Update `__init__.py`

To make sure `animation.py` is importable:

Python

```
"""
ASTE 404 Mini-Project Library
"""

from .solver import HybridSolver
from .nozzle import NozzleAnalyzer
from .animation import NozzleAnimator  # <--- NEW
from .utils import load_inputs, load_geometry
from .gas_dynamics import (
    area_mach_relation,
    normal_shock_relations,
    solve_expansion_fan,
    solve_oblique_beta
)

__version__ = "0.3.0"
```

How to Run It

1. Save the 3 files above.
2. Run the test script:
3. Bash

```
python test_step3.py
```

4.
5.

What you will see:

1. **Console:** A progress log counting up (0.0 km, 4.0 km, 8.0 km...) as it solves the math for every frame.
2. **Window:** A plot appears.
    - **Green Dashed Line:** Starts high (Sea Level) and drops rapidly.
    - **Blue Curve:** Initially has a sharp "step" (the shock) halfway through the nozzle.
    - **Red Line (Shock):** As the green line drops, the red shock line moves to the right.
    - **Pop:** Eventually, the shock hits the exit and disappears. The blue line becomes smooth (perfect supersonic flow).

are you able to access a github repo to check its contents

I cannot directly access or browse live websites, including GitHub repositories. I cannot "pull" code from a URL.

However, I can work with anything you share here. If you want me to check your progress or debug a specific file:

1. **Paste the code** directly into the chat (like you have been doing).
2. **Paste your file structure** (e.g., the output of `ls -R` or `tree`) if you want me to check if your folders are set up correctly.

Since we just finished **Step 3**, you should currently have this structure. If you want to verify, you can compare it against this list:

Plaintext

```
ASTE404FinalProject/
├── inputs/
│   ├── Klapsis_Nikitas_input.txt
│   └── Klapsis_Nikitas_nozzle_geometry.csv
├── src/
```

```
│   └── aste404_miniproject/
│       ├── __init__.py      (Updated to v0.3.0)
│       ├── animation.py     (New from Step 3)
│       ├── nozzle.py        (Contains NozzleAnalyzer class)
│       ├── solver.py        (Contains HybridSolver class)
│       ├── utils.py         (Loaders for txt/csv)
│       └── gas_dynamics.py  (Physics equations)
├── tests/
│   ├── test_step1.py
│   ├── test_step2.py
│   └── test_step3.py        (The one we just made)
└── pyproject.toml
```

If you are ready, we can move to **Step 4: The Design Tool** (Calculations to size a new engine).

how do I export this entire log

Since I am an AI, I cannot generate a downloadable file (like a `.txt` or `.pdf`) of our conversation for you directly. However, you can easily save this entire session yourself using one of these methods:

Method 1: Print to PDF (Best for Reading)

This preserves the formatting, images, and code blocks.

1. Press `Ctrl + P` (Windows/Linux) or `Cmd + P` (Mac).
2. Change the "Destination" or "Printer" to **"Save as PDF"**.
3. Click **Save**.

Method 2: Copy & Paste (Best for Code)

If you want to keep the code usable:

1. Click inside the chat window.
2. Press `Ctrl + A` (Select All) followed by `Ctrl + C` (Copy).
3. Open a blank document (Word, Google Docs, or a text file).
4. Paste it (`Ctrl + V`).

Did you mean the Python Output?

If you meant you want to save the **output of your script** (the flight data logs) to a file instead of just seeing it in the terminal, you can run this command in your terminal:

Bash

```
python test_step3.py > flight_log.txt
```

- This will run the script and save all printed text into `flight_log.txt`.
- Note: You won't see the text in the terminal while it runs, but the plot window will still pop up.